

Design and Verification of a Finite State Machine for a Newspaper Vending Machine

Major Project - Design, Simulation and Verification
By-Evan Abraham Bob

Problem statement

Design the control logic for a newspaper vending machine: the newspaper costs 15¢; the machine only takes nickels (N) and dimes (D); when a valid coin combination (total ≥ 15 ¢) has been inserted, it dispenses the paper and returns to its initial state; no change is given back, so a two-dime purchase (20¢) gives the paper and retains 5¢. Deliverables: an ASM chart, an HDL description, and a verified testbench. Interface: clk, active-high reset rst, coin inputs N and D (single-clock pulses, not occurring simultaneously), and output OPEN (single-cycle dispense strobe).

Design — deriving the state count

Only the credit accumulated in the current transaction. Any coin that makes the credit ≥ 15 ¢ immediately ends the transaction (dispense + reset), so the persistent credit values can only be (0¢, 5¢, 10¢)→ three states: S0, S1, S2. The retained 5¢ after a two-dime purchase (20¢) requires no additional state: it is accounting, not state, as it never influences any future behaviors — the next transaction starts at 0¢. The three states are pairwise distinguishable (from S1 a dime dispenses but from S0 it does not; from S2 a nickel dispenses but from S1 it does not; from S2 a dime dispenses but from S0 it does not), so the three-state design is minimal.

Present state	Input	Next state	OPEN	Why
S0 (0¢)	idle / N / D	S0 / S1 / S2	0	accumulate 5¢ or 10¢
S1 (5¢)	idle / N	S1 / S2	0	5¢ + 5¢ = 10¢
S1 (5¢)	D	S0	1	5 + 10 = 15¢ → dispense
S2 (10¢)	idle	S2	0	wait

Present state	Input	Next state	OPEN	Why
S2 (10¢)	N	S0	1	10 + 5 = 15¢ -> dispense
S2 (10¢)	D	S0	1	10 + 10 = 20¢ -> dispense, 5¢ retained

Minimized excitation equations (D flip-flops, K-maps with unused state 2'b11 as don't-care): $D1 = Q1'Q0'D + Q1'Q0N$, $D0 = Q1'Q0'N$, $OPEN = Q1'Q0D + Q1(N+D)$. The don't-care assignment makes the unused encoding 2'b11 fall safely to S0 in one cycle. A gate-level netlist built directly from these equations is included in the repository (vending_machine_structural.v) and behaves identically to the RTL.

Implementation

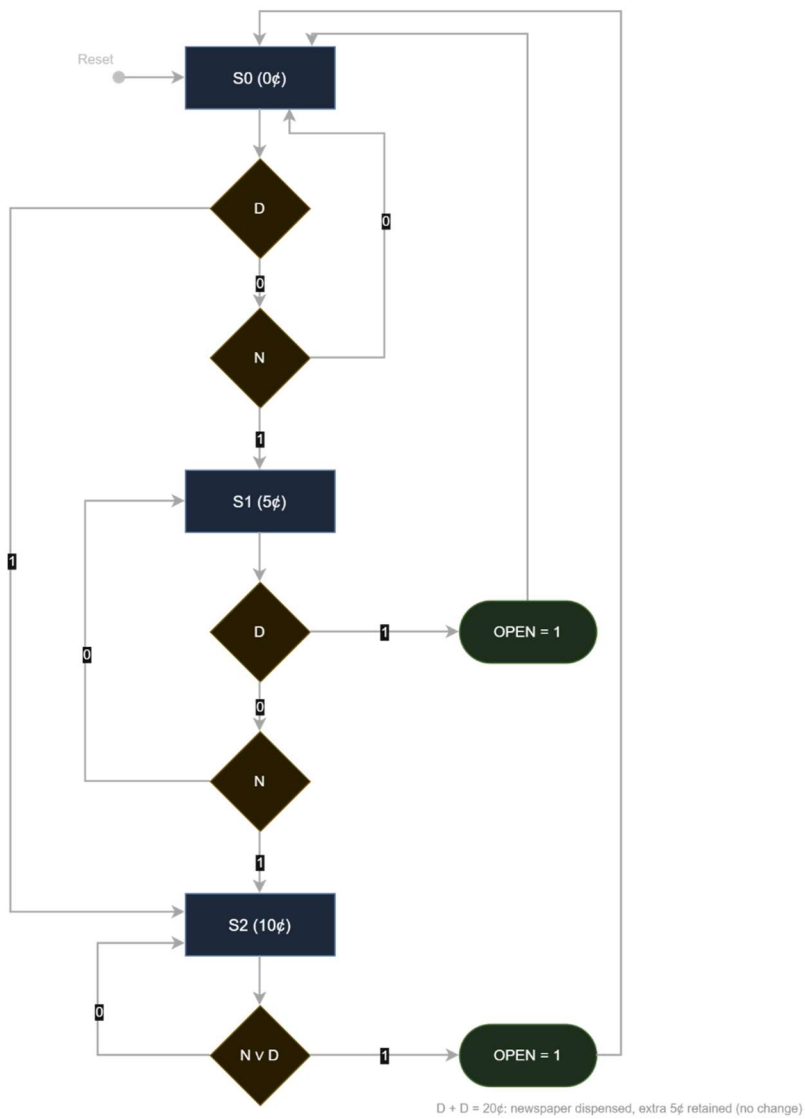
The Verilog module uses the standard three-block structure: an edge-triggered always block for the state register (asynchronous reset to S0), a combinational always block for the next-state logic with a default arm covering the unused encoding, and a continuous assignment for the Mealy output. OPEN asserts only in the cycle where the qualifying coin is present, so it is one clock wide by design. Full code listing: Appendix A; structural version: Appendix B.

Verification

The testbench is self-checking: a coin task drives each coin pulse and compares OPEN against the expected value, incrementing an error counter on mismatch, ending with an automatic verdict.

Test	Stimulus	Verifies	Result
T1	N, N, N (15¢)	first specified combination	PASS
T2	N, D (15¢)	second combination	PASS
T3	D, N (15¢)	third combination	PASS
T4	D, D (20¢)	overpayment: dispense once, no change	PASS

Test	Stimulus	Verifies	Result
T5	N, N, D (20¢)	generalized $\geq 15\text{¢}$ rule	PASS
T6	D, D, then N, D	new transaction starts at 0¢ (retained 5¢ ignored)	PASS
T7	reset mid-transaction, then D, N	reset recovery from any state	PASS



Simulation results:

```
[2026-09-05 07:58:06 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv && unbuffer vvp a.out
```

VCD info: dumpfile vending.vcd opened for output.

=== T1: N, N, N = 15c ===

[41] PASS: OPEN=0 (state=00)

[61] PASS: OPEN=0 (state=01)

[81] PASS: OPEN=1 (state=10)

=== T2: N, D = 15c ===

[101] PASS: OPEN=0 (state=00)

[121] PASS: OPEN=1 (state=01)

=== T3: D, N = 15c ===

[141] PASS: OPEN=0 (state=00)

[161] PASS: OPEN=1 (state=10)

=== T4: D, D = 20c (no change) ===

[181] PASS: OPEN=0 (state=00)

[201] PASS: OPEN=1 (state=10)

=== T5: N, N, D = 20c (>=15c rule) ===

[221] PASS: OPEN=0 (state=00)

[241] PASS: OPEN=0 (state=01)

[261] PASS: OPEN=1 (state=10)

=== T6: fresh transaction after D, D ===

[281] PASS: OPEN=0 (state=00)

[301] PASS: OPEN=1 (state=10)

[321] PASS: OPEN=0 (state=00)

[341] PASS: OPEN=1 (state=01)

=== T7: mid-transaction reset recovery ===

[361] PASS: OPEN=0 (state=00)

[411] PASS: OPEN=0 (state=00)

[431] PASS: OPEN=1 (state=10)

```
>>> ALL TESTS PASSED <<<
```

```
testbench.sv:68: $finish called at 440 (1ns)
```

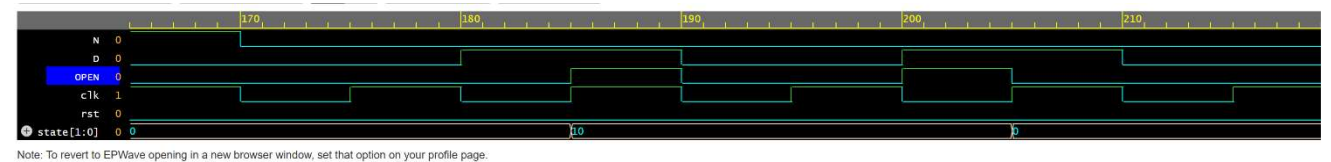
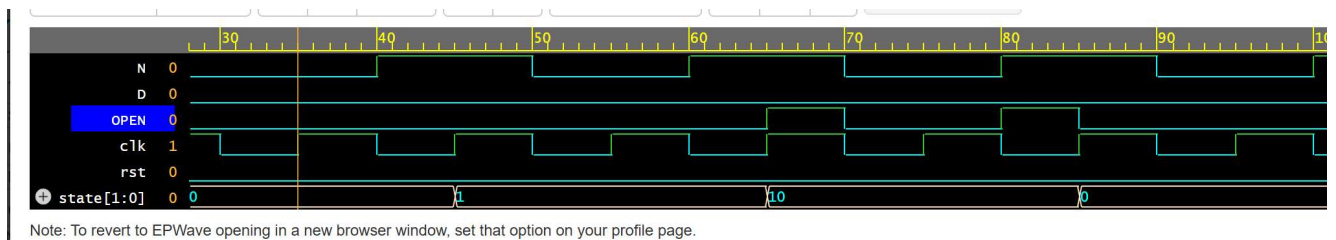
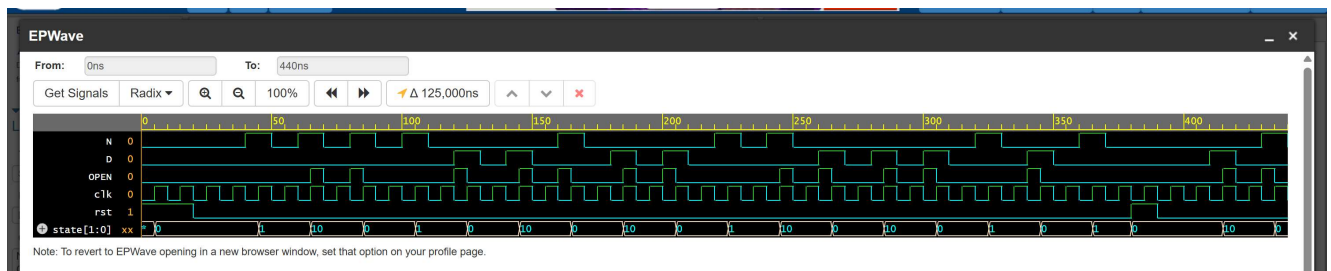
```
Finding VCD file...
```

```
./vending.vcd
```

```
[2026-09-05 07:58:07 UTC] Opening EPWave...
```

Done

Waveform:



Conclusion

A provably minimal three-state Mealy FSM controls the vending machine; a self-checking testbench covering all specified sequences, overpayment, generalized payment paths, and reset recovery passes with zero errors. The main transferable lesson: state count comes from which information about the past affects future behaviors — not from enumerating every possible coin sequence.

EDA PLAYGROUND LINK: - <https://edaplayground.com/x/Nkkd>

GitHub link: - <https://github.com/seriousorangeduck/majorproject>